



Please assist to provide me the following that is related to each individual Topic below, do not combine the topics, or lump them together:

1. Provide an overview of the @topic.
2. Develop thesis statements related to the @topic.
3. Conduct a literature review to support the thesis statements.

#####

@Topics

"15. Self-learning systems"

"20. Access control"

"19. Web security"

#####

It must follow and relate to the topic I'm researching for below.

#####

1. Securing the Open Banking & API Ecosystem

The financial world has shifted from isolated ecosystems to "Open Banking." Apps like Plaid, Venmo, and budgeting tools constantly pull data from traditional banks via APIs.

The Big Data Problem: Large banks handle billions of API calls daily, transmitting highly sensitive financial data across a web of third-party applications.

The Security Paradigm: Traditional firewall rules can't tell the difference between a legitimate third-party app requesting a user's balance and a hacker using stolen API keys or executing a "Broken Object Level Authorization" (BOLA) attack to scrape millions of accounts.

Thesis / Research Angle: Using Graph Neural Networks (GNNs) or streaming data analytics to model the entire API ecosystem. You could research how to detect anomalous data-exfiltration patterns or credential stuffing across billions of API logs in real time without lagging the user experience.

#####

Below are three separate research directions. Each one independently relates to your Open Banking/API ecosystem problem, but they differ in method, scope, and expected contribution.

15. Self-learning systems

Overview

In your Open Banking context, **self-learning systems** are AI/ML systems that learn normal and abnormal behavior from high-volume API events, rather than relying only on manually written firewall, WAF, or rate-limit rules.

The specific research use is to detect:

- BOLA-style cross-account access and object-ID enumeration.
- Credential stuffing followed by successful access to sensitive banking data.
- Stolen-token or stolen-API-key misuse.
- Compromised third-party provider (TPP) behavior.
- Low-and-slow data exfiltration that remains below static rate limits.
- New or evolving attack strategies that are absent from known attack signatures.

A suitable model is a **temporal heterogeneous graph**. It models several kinds of entities and relationships rather than treating each API log event independently:

$$G_t = (V_t, E_t, X_t)$$

Where:

- V_t represents nodes such as customers, financial accounts, consents, TPPs, OAuth clients, access tokens, IP addresses, devices, API endpoints, and requests.
- E_t represents time-stamped interactions such as “token issued to client,” “consent permits account,” “client accesses endpoint,” and “request retrieves account.”
- X_t represents features: request volume, HTTP status, token age, requested object, OAuth scope, response size, IP/ASN, device identity, request interval, endpoint sequence, and time of day.

For example, a valid TPP usually accesses only accounts connected through a customer’s consent. If that TPP’s token begins touching thousands of previously unrelated accounts, the individual API requests may appear valid, but the **graph structure** changes abnormally. This is a suitable case for temporal graph anomaly detection.

Financial-fraud research supports GNNs because relational methods capture network patterns and anomalies that flat, request-by-request models may miss. A systematic review of financial GNN research highlights gaps in unsupervised, edge-level, graph-level, heterogeneous, and temporal anomaly detection—areas directly relevant to detecting abnormal API-access relationships rather than only fraudulent transactions.^[1]

Thesis statements

Thesis Statement 15.1 — Recommended

A temporal heterogeneous graph-learning system that integrates Open Banking API telemetry with identities, OAuth tokens, consents, scopes, and account-access relationships can detect BOLA-style data exfiltration and credential abuse more effectively than request-local rules and tabular anomaly-detection baselines.

Thesis Statement 15.2 — Real-time focus

A streaming temporal graph anomaly-detection pipeline can identify low-and-slow account enumeration and anomalous API data-exfiltration patterns with lower time-to-detection than periodic batch analysis, while maintaining operationally acceptable API-service latency.

Thesis Statement 15.3 — Hybrid learning focus

A hybrid self-learning system combining deterministic API-security rules with graph-based anomaly scoring produces higher precision and lower false-positive rates for Open Banking API abuse than either static rules or machine learning alone.

Thesis Statement 15.4 — Authorization-aware AI

Adding consent, OAuth scope, token lineage, and account-ownership relationships to a temporal graph model improves detection of unauthorized Open Banking API access compared with models trained only on HTTP request metadata.

Literature review

GNNs for financial anomaly detection

GNNs are designed for data in which relationships are meaningful. Financial ecosystems naturally contain graph relationships among customers, accounts, devices, merchants, tokens, IP addresses, and transactions. API ecosystems add further relationships among consents, OAuth clients, endpoints, and object identifiers.

A systematic review of 33 selected studies on GNNs for financial fraud detection reports that GNNs can identify complex financial-network patterns that conventional rule-based and standard ML methods may fail to capture. It also notes limited research into unsupervised methods and edge-/graph-level anomalies, which is important because API abuse frequently occurs through abnormal access edges—for example, a token accessing an account outside its normal pattern—rather than an inherently malicious user node.^[1]

Graph-based anomaly-detection research has similarly found that connectivity patterns can reveal suspicious behavior that is difficult to detect from isolated records. This supports modeling API interactions as networks rather than as independent log rows.^[2]

Temporal graph learning

Open Banking requests are not static. A token may be normal at issuance and suspicious hours later. A TPP may gradually broaden the set of objects it accesses. A credential-stuffing campaign may generate a characteristic sequence of failed and successful authentication/API events.

Spatial-temporal attention GNNs have been applied to credit-card fraud detection, demonstrating that time and relationship structure can be jointly modeled for fraud decisions. More recent research continues to explore spatiotemporal GNNs for dynamic fraud networks, reinforcing the relevance of temporal interaction patterns for financial-risk detection.^{[3] [4]}

For your research, the key adaptation is:

Financial transaction graph → Open Banking authorization and API-access graph

The target is not only payment fraud. It is suspicious account-data access, consent misuse, token misuse, object enumeration, and data exfiltration.

Real-time and production constraints

Real-time GNN deployment is challenging because graph-neighbor queries and model inference can create significant latency. The BRIGHT framework addresses this issue by separating historical batch embeddings from real-time transaction scoring; its evaluation reported more than 75% lower p99 end-to-end latency and average 7.8× faster inference than a traditional GNN approach while improving average precision.^[5]

This is important for your thesis because “high accuracy” alone is insufficient. A practical Open Banking detector must be measured using:

- PR-AUC and ROC-AUC.
- Precision, recall, F1, and recall at a fixed false-positive rate.
- False alerts per million API requests.
- Detection delay from initial malicious access to alert.
- Throughput in events per second.
- p50, p95, and p99 latency.
- Memory/compute cost for graph-state maintenance.
- Robustness against low-and-slow attacks, token rotation, and distributed IP sources.

Streaming anomaly-detection research also identifies massive volumes of traffic and logs as a central practical challenge for cyber detection systems, particularly when the objective is identifying novel attacks quickly and efficiently.^[6]

Research gap

The literature has strong work on GNN-based transaction fraud and on streaming anomaly detection, but comparatively less work on **authorization-aware temporal graphs for Open Banking API misuse**. Your contribution can be to model:

$$\text{TPP} \rightarrow \text{OAuth client} \rightarrow \text{token} \rightarrow \text{consent} \rightarrow \text{account} \rightarrow \text{API endpoint}$$

and detect abnormal changes in these relationships at streaming scale.

20. Access control

Overview

In your research context, **access control** is the set of policies and enforcement mechanisms that determine whether a specific entity may perform a specific action on a specific banking resource.

The key question is:

Does this authenticated TPP, OAuth client, or token have permission to read or modify this exact customer account, transaction, balance, beneficiary, or payment instruction under this specific consent?

This is central to BOLA. A token may be valid, the request may be well formed, and the endpoint may be legitimate, yet the caller can still be unauthorized for the requested object.

A consent-aware Open Banking authorization decision can be represented as:

$$\text{Permit}(p, a, r, c, s, t, \gamma)$$

Where:

- p : principal, such as a TPP, OAuth client, user, or internal service.
- a : action, such as `read_balance`, `read_transactions`, or `initiate_payment`.
- r : protected object, such as a specific account or transaction.
- c : valid customer consent.
- s : OAuth scope or granted permission.
- t : time conditions, expiry, frequency, or token status.
- γ : contextual attributes, such as device, IP, client certificate, risk score, location, or transaction value.

The system should deny an action unless the policy proves that the access relationship is valid. This is usually called a **default-deny** or **deny-by-default** approach.

OWASP identifies API1:2023 as Broken Object Level Authorization and explains that API endpoints often handle object identifiers, creating a broad attack surface. It recommends

object-level authorization checks in every function that accesses data through a user-provided identifier.^[7]

Thesis statements

Thesis Statement 20.1 — Recommended

A consent-aware, relationship-based access-control model for Open Banking APIs can prevent a broader class of BOLA attacks than endpoint-level role-based access control by evaluating the requester, token, consent, scope, action, and target account as a single authorization decision.

Thesis Statement 20.2 — Policy-verification focus

Formal verification and automated testing of object-level authorization policies can identify cross-account access paths in Open Banking APIs before deployment, reducing the risk of BOLA vulnerabilities caused by inconsistent resource-server enforcement.

Thesis Statement 20.3 — Adaptive access-control focus

Risk-adaptive access control that incorporates contextual signals such as token behavior, device changes, IP reputation, and anomalous account-access patterns can reduce unauthorized Open Banking data access while preserving legitimate third-party access.

Thesis Statement 20.4 — Relationship-based authorization

Relationship-based access control models provide a more accurate representation of Open Banking consent relationships than traditional RBAC because authorization depends on dynamic links among customers, TPPs, consents, accounts, scopes, and delegated permissions.

Literature review

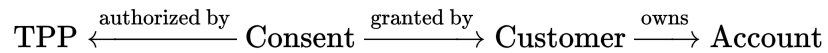
Traditional access-control models

Classic access-control approaches include:

- **Discretionary Access Control (DAC):** resource owners control permissions.
- **Mandatory Access Control (MAC):** centrally enforced classifications and policies govern access.
- **Role-Based Access Control (RBAC):** permissions are assigned to roles, such as customer, TPP, bank administrator, or support agent.
- **Attribute-Based Access Control (ABAC):** decisions depend on attributes of the subject, resource, action, and environment.
- **Relationship-Based Access Control (ReBAC):** decisions depend on relationships, such as whether an account is connected to a valid consent granted to a particular TPP.

RBAC is useful for broad role separation, but it is insufficient as the only decision mechanism for Open Banking. A “TPP” role does not establish that the TPP may access any customer’s data. The required authorization depends on a precise relationship: the customer’s consent must link the TPP, client, permissions, and specified account/resource.

ABAC supports dynamic attributes: scope, token expiry, geography, device, consent status, transaction amount, or risk level. ReBAC is especially natural for delegated financial access because it expresses the required path:



BOLA as the primary threat

OWASP’s API Security Top 10 places BOLA first. It occurs when an API accepts a caller-supplied object identifier but does not adequately validate that the requester is authorized to access that exact object. ^[8] ^[7]

A simple example is:

```
GET /v1/accounts/982734/transactions
Authorization: Bearer <valid_token>
```

A resource server must not only validate the token. It must evaluate whether:

- The token belongs to the identified client.
- The customer consented to this access.
- The consent is active and unexpired.
- The consent authorizes transaction access.
- Account 982734 is included in that consent.
- The requested operation and field set are permitted.
- Contextual restrictions are satisfied.

OWASP’s API Security Testing Framework includes BOLA testing through ID manipulation and cross-user access confirmation using two distinct identities. This provides a practical evaluation tool for a prototype API implementation. ^[9]

Financial-grade API standards

Open Banking commonly uses OAuth 2.0 and OpenID Connect, with FAPI adding high-security constraints appropriate for valuable financial data. FAPI 2.0 is an OAuth-based API-security profile suitable for high-security applications, and its security objectives include preventing attackers from accessing resources that belong to others. ^[10] ^[11]

FAPI supports a strong baseline for your access-control research:

- TLS-protected API communication.

- Authenticated clients.
- OAuth-protected resource endpoints.
- Customer identity via OpenID Connect.
- Token-based access to protected financial resources.
- Fine-grained and transactional authorization capabilities.
- Replay-detection and non-repudiation mechanisms in relevant profiles. ^[12] ^[13]

However, protocol-level security does not guarantee correct resource-server authorization logic. A system can correctly issue and validate OAuth tokens but still expose another customer's account if object-level checks are inconsistent, incomplete, or bypassable.

Research gap

Existing standards define strong authentication and authorization flows, while OWASP defines BOLA and associated API risks. The research opportunity is to propose and evaluate an **authorization model or enforcement architecture** that operationalizes consent at object level.

A high-value contribution would be a policy engine that evaluates:

$\text{Allow} \iff \text{ValidToken} \wedge \text{ValidConsent} \wedge \text{PermittedScope} \wedge \text{ObjectInConsentSet} \wedge \text{ContextAI}$

Then test it with legitimate access, expired consent, scope escalation, token replay, cross-account ID substitution, and compromised-TPP scenarios.

19. Web security

Overview

For your topic, **web security** means protecting the Open Banking API layer—the HTTP/HTTPS interfaces through which TPPs, mobile applications, aggregators, and internal services request financial data or initiate financial actions.

This includes security controls at:

- API gateways and reverse proxies.
- OAuth authorization servers.
- Resource servers exposing account, transaction, payment, or identity APIs.
- Web/mobile client applications.
- Third-party integrations.
- Logging, SIEM, and security-operations pipelines.
- CI/CD pipelines and API inventories.

The research problem is that conventional web controls are usually **request-local**. A WAF can identify malformed payloads; a gateway can apply quotas; OAuth can verify an access token; and a firewall can enforce network boundaries. But none necessarily understand that a valid

token accessing an unusual set of accounts over time may indicate data exfiltration or a compromised third party.

OWASP's 2023 API Security Top 10 defines the principal API-specific threat surface:

OWASP API risk	Relevance to your Open Banking research	
API1: BOLA	Cross-account reads via manipulated account or transaction identifiers	
API2: Broken Authentication	Credential stuffing, stolen API keys, token theft, session/token misuse	
API3: Broken Object Property Level Authorization	Excessive exposure of financial data fields	
API4: Unrestricted Resource Consumption	Scraping, excessive pagination, resource exhaustion	
API5: Broken Function Level Authorization	Unauthorized use of privileged payment/admin functions	
API6: Unrestricted Access to Sensitive Business Flows	Automated abuse of account opening, payments, beneficiary creation, or data exports	
API7: SSRF	Attacks that make a server call internal or external resources	
API8: Security Misconfiguration	Overly permissive CORS, exposed debug endpoints, weak TLS, bad gateway policy	
API9: Improper Inventory Management	Forgotten APIs, old versions, shadow endpoints	
API10: Unsafe Consumption of APIs	Risks from relying on untrusted/compromised third-party API data	[8] [7]

Thesis statements

Thesis Statement 19.1 — Recommended

A behavior-aware API gateway that combines Open Banking security controls with streaming anomaly detection can identify and mitigate credential stuffing and account-data exfiltration more effectively than conventional request-rate limits and signature-based web-application defenses.

Thesis Statement 19.2 — API telemetry focus

Enriching API-gateway telemetry with OAuth, consent, client, endpoint, and response-volume metadata improves the detection of BOLA-style enumeration and bulk financial-data extraction compared with HTTP request metadata alone.

Thesis Statement 19.3 — Defense-in-depth focus

A defense-in-depth Open Banking web-security architecture that combines FAPI-compliant authorization, API schema validation, object-level checks, token protection, rate controls, and behavioral monitoring reduces exposure to the OWASP API Top 10 more effectively than perimeter controls alone.

Thesis Statement 19.4 — Third-party integration focus

Continuous behavioral monitoring of third-party API clients can reduce the risk of unsafe API consumption and compromised-TPP data exfiltration by identifying deviations from approved consent, endpoint, account, and response-volume patterns.

Literature review

Web security versus API security

Traditional web security focuses on browser applications, sessions, HTML injection, cross-site scripting, CSRF, and server-side vulnerabilities. API security overlaps with it but is distinct because APIs expose machine-readable, high-value business objects and are commonly accessed programmatically at scale.

In Open Banking, APIs are the product boundary: they expose account balances, transactions, customer information, payment initiation, consent workflows, and financial functions to approved third parties. Consequently, web security must protect not only the API's HTTP syntax but also its business rules, resource relationships, and data-flow semantics.

The OWASP API Top 10 recognizes this difference. It highlights BOLA, broken authentication, object-property authorization, resource exhaustion, sensitive-business-flow abuse, misconfiguration, inventory failures, and unsafe third-party API consumption.^[8]

Financial-grade API protection

FAPI is directly relevant because it was developed for high-security financial API interactions. The FAPI 2.0 Baseline Profile describes itself as an OAuth-based API-security profile suited to high-value scenarios, including sensitive personal data. It requires TLS-protected endpoints and defines OAuth-protected resource endpoints that return protected information associated with the resource owner of the access token.^{[14] [12]}

The FAPI Working Group describes FAPI as a general-purpose high-security API profile over OAuth and states that FAPI 2.0 supports fine-grained/transactional authorization, replay detection, and non-repudiation mechanisms.^[13]

For your thesis, FAPI should be treated as a **security baseline**, not as the research result. It provides protocol security; your research asks how operational security monitoring can detect abnormal use despite technically compliant HTTP, TLS, and token flows.

Limitations of request-local controls

Conventional controls remain necessary:

- TLS protects communications in transit.
- OAuth/OIDC authenticates users and authorizes client access through tokens.
- API gateways validate requests and enforce quotas.
- WAFs block known patterns and common exploit attempts.
- Rate limiting restrains obvious brute-force and volumetric abuse.

- API schema validation rejects malformed requests.
- Logging provides evidence for detection and investigation.

But these controls may miss valid-looking malicious activity:

- A credential-stuffing attacker uses distributed IPs and slow rates.
- A stolen token is used correctly at the protocol level.
- A compromised TPP legitimately authenticates but violates expected data-access behavior.
- An account enumeration attack stays just below static quota thresholds.
- A BOLA defect permits object access because the resource server applies authentication without object-level authorization.

This limitation motivates behavior-aware API protection. The goal is not to remove gateway rules but to make them **risk-adaptive**. A system may allow low-risk traffic, monitor medium-risk traffic, apply step-up verification at high risk, and deny/revoke credentials at critical risk.

Testing and validation

OWASP's API Security Testing Framework offers automated testing for API vulnerabilities mapped to the 2023 Top 10. Its BOLA test coverage includes object-ID manipulation and cross-user access confirmation using two separate identities. ^[9]

That gives you a practical experimental method:

1. Create a mock Open Banking API with accounts, transactions, OAuth tokens, and consent records.
2. Populate valid TPP/client access paths.
3. Generate attack cases: BOLA ID substitution, excessive pagination, credential stuffing, stolen-token replay, scope abuse, and compromised-client exfiltration.
4. Apply baseline controls: OAuth validation, basic authorization, WAF/rate limits.
5. Evaluate whether enriched telemetry and behavioral detection reduce undetected attack activity.

Research gap

OWASP and FAPI provide essential security guidance, protocol controls, and testable requirements. However, they do not themselves provide a complete solution for detecting **valid-but-malicious behavioral patterns** across billions of API calls.

A focused web-security contribution would therefore be:

Designing and evaluating a behavior-aware API-gateway architecture that enriches conventional Open Banking web controls with streaming telemetry analysis for detecting BOLA-like enumeration, credential stuffing, and anomalous bulk account-data access.

This research direction is feasible if you keep the scope focused on a small but realistic API surface: account balance retrieval, transaction retrieval, OAuth consent, token issuance/refresh, and a simulated TPP client.

✱

1. <https://dl.acm.org/doi/10.1016/j.eswa.2023.122156>
2. <https://dl.acm.org/doi/10.1016/j.dss.2020.113303>
3. <https://ieeexplore.ieee.org/document/9204584/>
4. <https://ieeexplore.ieee.org/document/11426159/>
5. <https://dl.acm.org/doi/10.1145/3511808.3557136>
6. <https://pubmed.ncbi.nlm.nih.gov/40989415/>
7. <https://owasp.org/API-Security/editions/2023/en/0x11-t10/>
8. <https://owasp.org/API-Security/editions/2023/en/0x00-toc/>
9. <https://owasp.org/www-project-api-security-testing-framework/>
10. https://openid.net/specs/fapi-2_0-security-profile-ID2.html
11. https://openid.net/specs/fapi-2_0-attacker-model.html
12. https://openid.net/specs/fapi-2_0-baseline.txt
13. <https://openid.net/wg/fapi/>
14. https://openid.net/specs/fapi-2_0-baseline-01.html
15. https://openid.net/wordpress-content/uploads/2022/12/Formal-Security-Analysis-of-FAPI-2.0_FINAL_2022-10.pdf
16. https://owasp.org/www-chapter-bangkok/slides/2023/2023-03-31_OWASP-API.pdf
17. <https://ieeexplore.ieee.org/iel8/8782664/10834807/10892045.pdf>
18. <https://ieeexplore.ieee.org/iel8/11425842/11425023/11426159.pdf>
19. <https://dl.acm.org/doi/abs/10.1016/j.eswa.2021.116463>
20. <https://ieeexplore.ieee.org/abstract/document/10903430>
21. <https://dl.acm.org/doi/10.1145/3442381.3449989>
22. <https://dl.acm.org/doi/10.1145/3641283>
23. <https://ieeexplore.ieee.org/iel8/11340590/11341918/11342193.pdf>
24. <https://ieeexplore.ieee.org/document/11070932/>
25. https://openid.net/specs/openid-financial-api-part-1-1_0-11.html
26. <https://owasp.org/API-Security/editions/2023/en/0x03-introduction/>
27. <https://owasp.org/API-Security/editions/2023/en/0x04-release-notes/>
28. <https://owasp.org/blog/2023/07/03/owasp-api-top10-2023>
29. <https://owasp.org/API-Security/editions/2023/en/0x00-notice/>
30. <https://dl.acm.org/doi/10.1007/s11704-024-40474-y>